Runbook — Deploy Captain Adel (the RAG chat gateway)

SECTION 06-operations-it TYPE runbook STATUS active OWNER Founder UPDATED 2026-08-19

Captain Adel is the **brain behind** `/chat`: it retrieves GACAR passages and asks Gemini for an answer, cited to the exact Part/section. It is not a separate deployable — it is a set of routes inside the one Express service on Cloud Run (`/api/chat` and `/api/feedback`, plus the licensed `/v1/ask` surface). Deploying Captain Adel means deploying the API.

> [!NOTE] This page is about Captain Adel **inside the product** (`iflygaca/FlyGACA`). The standalone service behind `captadel.com` lives in `iflygaca/Captain-Adel` and has its own `runbook-captadel-*` pages in this folder. Do not mix the two.

What you need first

1. **A Gemini API key** — aistudio.google.com → *Get API key*. It is stored in Secret Manager as `genai-api-key` and reaches the service as `GOOGLE_GENAI_API_KEY`. Never committed.
2. **The RAG corpus built into the image.** `npm run build:chunks` regenerates `public/data/rag-chunks.json`; the Dockerfile copies it into the image so the BM25 index needs no cold-start fetch.
3. **The Cloud Run service itself** — provisioned per `docs/RUNBOOK-deploy.md` in `iflygaca/FlyGACA`. There is no Firebase, no Cloud Functions and no Blaze plan involved.

Deploy

There is no chat-specific deploy command. Rebuild the corpus if it changed, then deploy the API exactly as the product-repo runbook describes (`gcloud run deploy` from the repo root, with the secrets and prices set on the revision).

`assertRequiredConfig()` runs before the listener binds, so a revision missing `DATABASE_URL` or carrying a too-short `SESSION_SECRET` fails its health check instead of 500-ing later. `GET /healthz` returns 503 until the database answers.

Once the revision is live, ask a GACAR question on `/chat` — for example *"What are the VFR weather minima?"*. The answer should carry **Sources** that deep-link into the library reader.

How it works

```
/chat (SPA)  ──POST /api/chat──▶  Cloud Run (Express 5)
                                          │
                            gateway.ts    │  quota + rate limits + SSE framing
                                          │
                            corpus.ts     │  BM25 over rag-chunks.json
                                          │
                          captain-adel.ts │  Genkit → Gemini, cited answer
```

Two design points that matter operationally:

- **Grounding is computed server-side, not asked of the model.** Below the refusal threshold the model is *not called at all* — the user gets a cite-the-rule refusal instead. So a fabricated GACAR figure cannot be emitted just because the model was confident.
- **The system guardrails are unit-tested.** The not-affiliated / educational-only rules live in their own module with a test that fails the build if one is dropped. They are the server-side twin of the site-wide disclaimer — treat them like the disclaimer component: don't reword them ad hoc.

Tuning & cost control

Lever	Where	Default
Model (free tier)	Gemini via Genkit	`gemini-2.5-flash`
Model (Pro tier)	selected per request from the caller's plan	`gemini-2.5-pro`
Passages retrieved	`RETRIEVE_K`	`6`
Refuse-below score	`REFUSE_SCORE`	`1.5`
Fully-grounded score	`GROUNDED_SCORE`	`4`
Free questions per day	`CHAT_FREE_DAILY_LIMIT`	`5`
Questions per purchased credit pack	`CHAT_CREDIT_PACK_SIZE`	`50`
Chat kill switch	`CHAT_ENABLED`	`true`
Burst limit per account	in `gateway.ts`	20 turns/min
Coarse limit per IP	in `gateway.ts`	30 requests/min
Licensed API limit per key	in `gateway.ts`	60 requests/min

The daily quota is the real cost control and it is **per account, per day** — the free tier is five questions a day, and the client-side mirror of that policy is test-enforced against the server core, so the two cannot drift. The burst limiters are per-instance memory: they guard against runaway loops and casual abuse, not a distributed attack.

`CHAT_ENABLED=false` is the emergency stop if model spend runs away.

Updating later

- **Changed the corpus?** `npm run build:chunks`, commit the regenerated `public/data/rag-chunks.json`, redeploy the API.
- **Changed Captain Adel's persona?** Edit the prompt module in `server/src/`, run the server test suite (the guardrail test will tell you if you removed a rule), redeploy.
- **Rotated the Gemini key?** Add a new version to the `genai-api-key` secret and deploy a new revision — secrets are read at revision start.
- **Changed pricing or quota policy?** The server core is the source of truth; the client mirror test in the product repo fails the build if you update one and not the other.

Troubleshooting

Symptom	Likely cause
Chat returns a refusal for a question the library clearly answers	Retrieval scored below `REFUSE_SCORE` — check the corpus was rebuilt after the last content sync.
Every request 503s	The service is up but the database is not answering — `/healthz` gates on it.
`429 rate_limited`	Burst limiter hit (20 turns/min per account). Expected under scripted load.
Quota exhausted for a paying user	Check the entitlement row — plan and credits are server-owned and written only by the billing and grants routes.
Model errors / quota from Google	Gemini API limits on the key's project. Raise the quota or lower traffic.
Answers arrive without citations	Corpus file missing from the image (`CORPUS_URL`) — the Dockerfile copy step.

Pre-launch reminders

- Captain Adel is an **educational AI**. The on-page disclaimer and the server-side guardrails both say so. Keep both.
- The corpus is GACA-published material only. Live NOTAMs and weather are deliberately out of scope, and the prompt declines them.
- Confirm the GACAR redistribution position with Saudi counsel (P0-1) before a public launch.